

4교시: Grafana Dashboard 확인

W4D3 Lesson 4 Grafana dashboard

Kubernetes 클러스터의 리소스와 애플리케이션 상태를 한눈에 관찰하고 문제를 빠르게 파악합니다.

1. 관찰 스택 구조

2. kubectl 명령과 Grafana 연계

kubectl로 현재 상태 확인 → Grafana로 과거 추이 분석

리소스 사용량 (실시간)

```
$ kubectl top pod -n app-ns
```

	NAME	CPU(cores)	MEMORY(bytes)
front-end	front-5d7f8dc7f-abc12	15%	964M
api	api-7c9d6f8f9b-def34	28%	120M

파드 상태 확인

```
$ kubectl get pod -n app-ns
```

NAME	READY	STATUS	RESTARTS	AGE
front-end	1/1	Running	0	2d
api	1/1	Running	1	2d

노드 리소스 확인

```
$ kubectl top node
```

NAME	CPU(cores)	MEMORY(bytes)	MEMORY%
node-1	42%	3.16Gi	53%
node-2	31%	3.46Gi	41%

4. Grafana 대시보드 예시

시간 범위 선택

- Last 5m / 15m / 1h / 6h / 24h / 7d / Custom
- 자동 새로고침(30s) 으로 실시간 모니터링

팁: 시간 범위를 보아서 장애 전후의 패턴을 확인하고, 변수가 같은 대시보드를 만들어 빠르게 원하는 대상을 필터하세요.

3. 핵심 지표 설명

- CPU 사용량 (cores)**: 컨테이너/노드의 CPU 사용량 추이
- 메모리 Working Set (bytes)**: 사용 중인 메모리(가비지 컬렉션 제외)
- Restarts**: 파드 재시작 횟수 추이
- Ready Replicas**: 배포된 레플리카 중 Ready 상태 수

5. 대시보드 예시 목록

대시보드 이름	설명
Kubernetes / Namespace	네임스페이스 커널 리소스 및 파드 상태
Kubernetes / Pod	파드 단위의 리소스 사용량 및 상태
Kubernetes / Node Exporter	노드(서버) 리소스 모니터링
Ready Replicas	클러스터 전체 요청 Ready 상태 미러링

6. 알람(Alert) 예시

조건	설명
Pod Restart > 3 (5m)	짧은 시간 내 파드 반복 재시작
CPU Usage > 80% (5m)	지속적인 CPU 과다 사용
Memory Usage > 85% (5m)	메모리 사용량 과다
Not Ready Pods > 0 (2m)	Ready 상태가 아닌 파드 존재

7. 추가 참고

- Provisioning으로 대시보드/데이터소스 자동 배포 가능
- 라벨/주석을 활용한 그룹화 및 필터링
- Annotation으로 배포 이벤트를 함께 표시
- Tempo/Jaeger 연동으로 트레싱 분석

시간 범위와 변수를 적절히 활용하면 이상 징후를 빠르게 발견할 수 있어요!

★ Best Practice

- ✓ 대시보드는 용도별로 분리
- ✓ 적절한 변수와 템플릿 사용
- ✓ 알람은 의미 있는 임계값 설정
- ✓ 장애 대응 Runbook과 연계

수업 목표

- Grafana에서 Kubernetes dashboard를 찾고 읽는다.
- node/pod CPU, memory, restart, namespace resource 사용량을 확인한다.
- dashboard 수치와 kubectl 명령을 연결한다.

Grafana 접속

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

브라우저:

http://localhost:3000

로그인:

```
admin / paperclip-local
```

먼저 볼 dashboard

Dashboard keyword	질문
Kubernetes / Compute Resources / Namespace	어느 namespace가 자원을 쓰는가
Kubernetes / Compute Resources / Pod	어느 Pod가 CPU/memory를 쓰는가
Kubernetes / Kubelet	kubelet과 Pod resource
Node Exporter	node 전체 CPU/memory/disk/network

kubectl과 dashboard 연결

```
kubectl top node
kubectl top pod -A
kubectl -n week4 get pod
```

Grafana는 같은 정보를 시간 축으로 보여준다.

kubectl	Grafana
현재 CPU/memory	시간별 CPU/memory
현재 restart count	restart 증가 시점
현재 Pod READY	ready replica 추세
describe event	metric 변화와 함께 원인 추적

dashboard를 보는 순서

처음부터 모든 panel을 보려고 하면 길을 잃는다.

순서	질문	dashboard
1	node 전체가 힘든가	Node Exporter
2	어떤 namespace가 쓰는가	Compute Resources / Namespace
3	어떤 Pod가 쓰는가	Compute Resources / Pod
4	restart가 늘었는가	Workload/Pod dashboard
5	target이 정상인가	Prometheus Targets

이 순서는 “전체 -> namespace -> pod -> 원인”으로 좁히는 방식이다.

PromQL 예시

namespace별 memory:

```
sum by (namespace) (container_memory_working_set_bytes{container!="", image!=""})
```

Pod별 CPU:

```
sum by (namespace, pod) (rate(container_cpu_usage_seconds_total{container!="", image!=""}[5m]))
```

restart:

```
increase(kube_pod_container_status_restarts_total[10m])
```

Ready replica:

```
kube_deployment_status_replicas_ready
```

그래프 시간 범위

Grafana 오른쪽 위 time range를 조정한다.

범위	용도
Last 5 minutes	방금 재현한 실습 장애
Last 30 minutes	rollout 전후 비교
Last 1 hour	수업 전체 흐름 확인
Last 6 hours	retention 안에서 넓게 보기

실습 장애가 보이지 않으면 time range가 너무 넓거나 좁은지 먼저 본다.

dashboard와 alert의 차이

Dashboard는 사람이 보는 화면이고, alert는 사람이 보지 않아도 알려주는 조건이다.

dashboard

-> 사람이 들어가서 관찰

alert

-> 조건이 맞으면 사람에게 알림

그래서 dashboard에서 보이는 모든 spike를 alert로 만들면 안 된다.

dashboard 해석 주의

실수	문제
순간 spike만 보고 장애 판단	지속 시간과 사용자 영향 확인 필요
CPU만 봄	memory, restart, readiness도 함께 확인
node 전체만 봄	namespace/Pod 단위로 좁혀야 함
dashboard만 봄	logs/events로 원인 확인 필요

dashboard에서 흔히 보는 패널 해석

패널	질문	다음 행동
CPU Usage	CPU를 많이 쓰는 Pod는 무엇인가	<code>kubectl top pod</code> , limit 확인
Memory Working Set	memory가 계속 증가하는가	OOMKilled event 확인
Pod Restarts	재시작이 늘었는가	<code>logs --previous</code> 확인
Ready Replicas	배포가 준비됐는가	readiness event, endpoint 확인
Network Receive/Transmit	traffic이 증가했는가	Ingress/API 요청량과 비교

패널 하나만 보고 결론을 내리지 않는다. CPU가 높아도 정상 batch일 수 있고, restart가 1회 있어도 rollout 중 정상일 수 있다.

Grafana에서 label로 좁히기

dashboard 상단에는 cluster, namespace, workload, pod 같은 variable이 있다. 먼저 namespace를 좁히고, 다음에 workload/pod를 좁힌다.

```
All namespaces
-> week4
-> api
-> api-xxxxx
```

이 순서를 지키면 전체 cluster 문제인지 특정 서비스 문제인지 구분할 수 있다.

legend가 지저분할 때

Grafana panel의 legend가 container, endpoint, instance, job, namespace, pod, service label을 모두 달고 나오면 사람이 읽기 어렵다. 이때는 query에서 필요한 label만 남기거나, Grafana legend format을 조정한다.

Pod별 CPU를 볼 때:

```
sum by (pod) (
  rate(container_cpu_usage_seconds_total{namespace="week4-observe", container!="",
image!=""}[2m])
)
```

container까지 보고 싶을 때:

```
sum by (pod, container) (
  rate(container_cpu_usage_seconds_total{namespace="week4-observe", container!="",
image!=""}[2m])
)
```

legend 예:

```
{{pod}}
{{pod}} / {{container}}
```

수업에서는 먼저 namespace, pod, container 정도만 남긴다. instance나 endpoint는 Prometheus가 어디
서 scrape했는지를 보여주는 운영 label이라, 처음 dashboard를 읽는 학생에게는 noise가 될 수 있다.

수업 중 확인할 최소 세 가지

확인	이유
monitoring namespace Pod resource	stack 자체가 건강한가
week4 또는 week4-observe namespace resource	실습 앱 영향
restart panel	CrashLoop 시나리오가 metric에 보이는가

dashboard와 screenshot 증거

배움일기에 screenshot을 넣는다면 예쁘게 캡처하는 것보다 아래 정보가 보여야 한다.

포함할 것	이유
time range	언제의 문제인지
namespace/pod filter	무엇을 보고 있는지
panel title	어떤 지표인지
spike 또는 변화 구간	장애 시점
같은 시점의 kubectl evidence	metric과 원인 연결

Evidence Note

W4D3S4 Grafana

- 접속 URL:
- 확인한 dashboard:
- CPU가 높은 namespace/pod:
- memory가 높은 namespace/pod:
- restart가 보인 Pod:
- kubectl과 비교한 명령:
- dashboard time range:
- screenshot에 포함한 정보:

한 줄 요약

Grafana dashboard는 kubectl의 현재 상태를 시간 축으로 확장해 장애의 시작점과 범위를 찾게 해준다.